

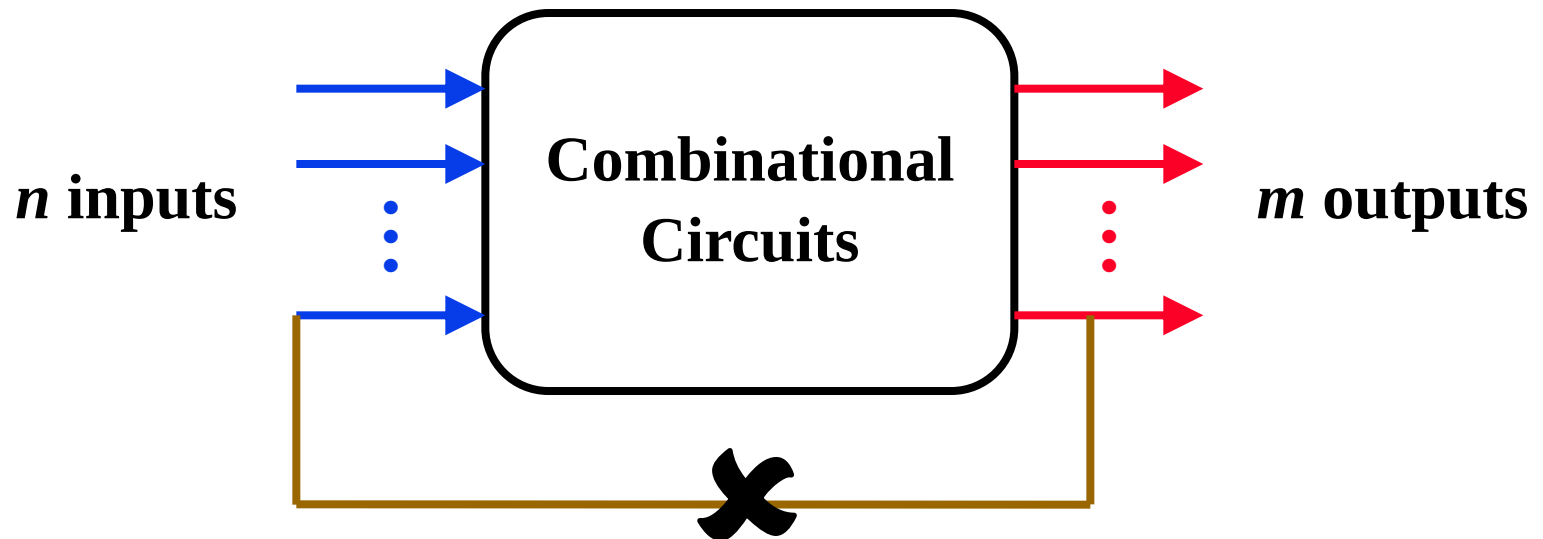
Combinational Logic

Prof. Dr. Khalid M. Hosny

Combinational Circuits

★ Output is function of input only

i.e. no feedback

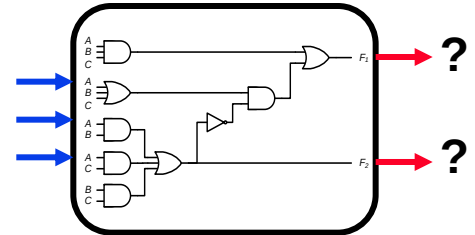


When **input** changes, **output** may change (after a delay)

Combinational Circuits

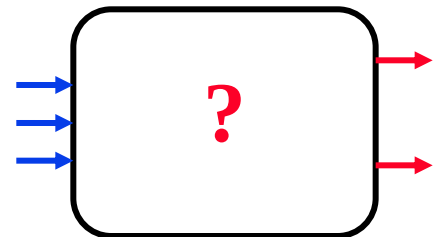
★ Analysis

- Given a circuit, find out its *function*
- Function may be expressed as:
 - ◆ Boolean function
 - ◆ Truth table



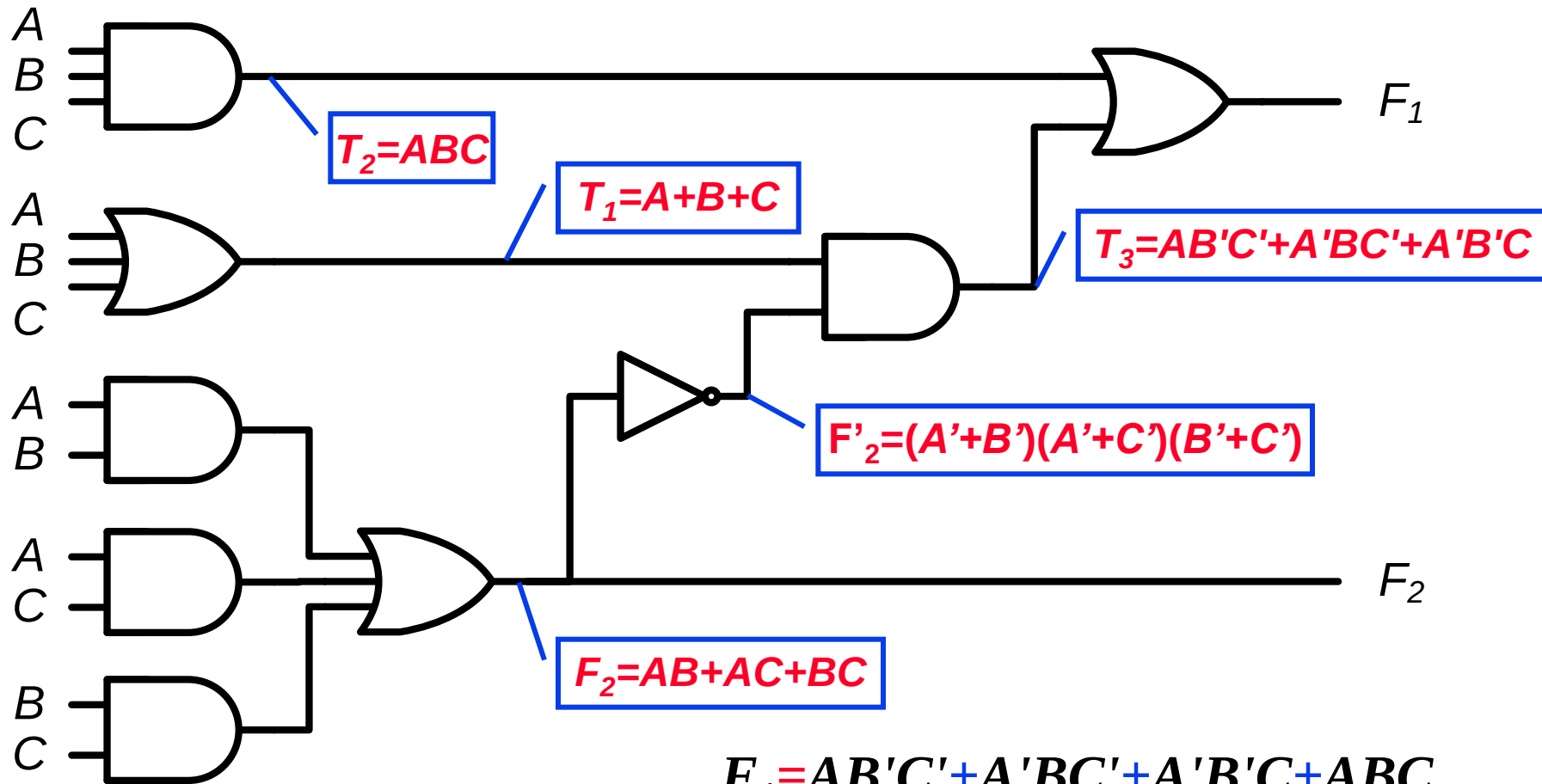
★ Design

- Given a desired function, determine its *circuit*
- Function may be expressed as:
 - ◆ Boolean function
 - ◆ Truth table



Analysis Procedure

★ Boolean Expression Approach

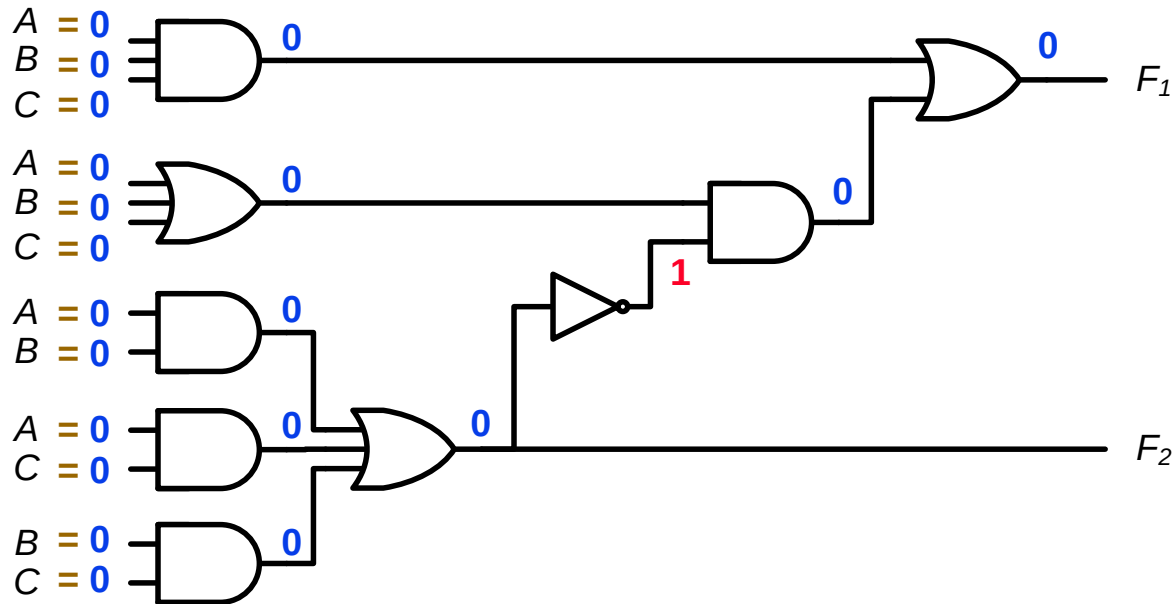


$$F_1 = AB'C' + A'BC' + A'B'C + ABC$$

$$F_2 = AB + AC + BC$$

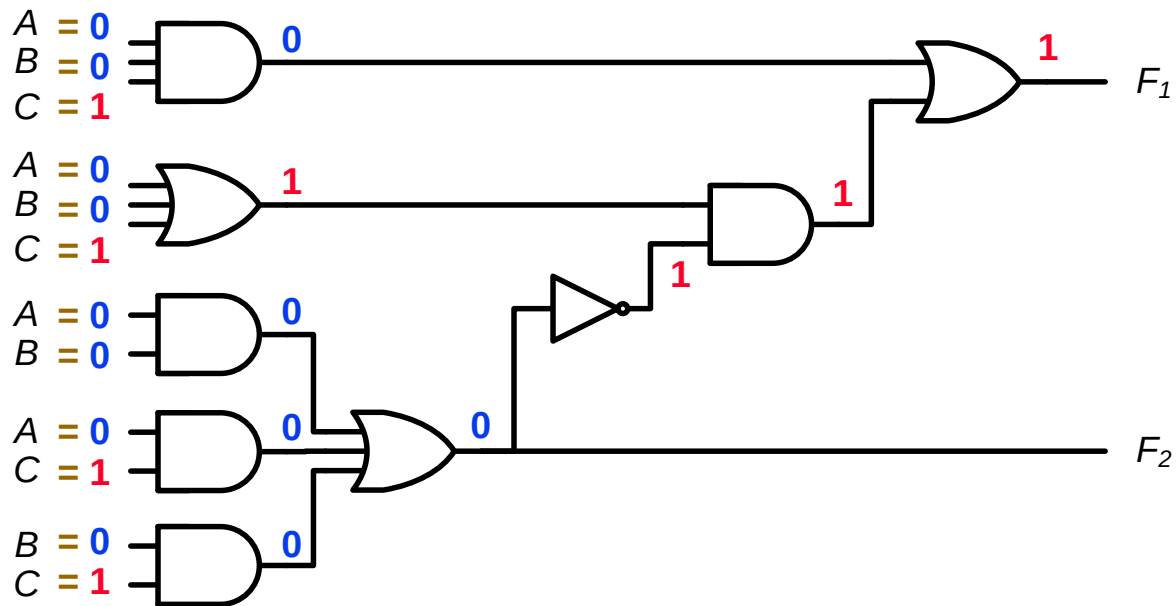
Analysis Procedure

★ Truth Table Approach

[illegible]

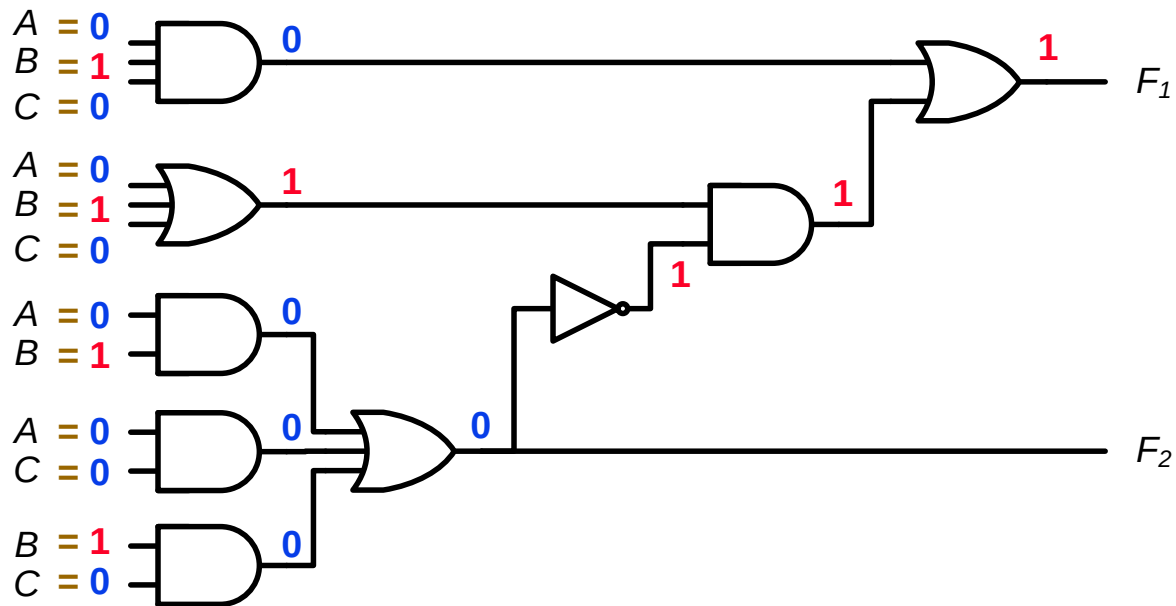
Analysis Procedure

★ Truth Table Approach

[illegible]

Analysis Procedure

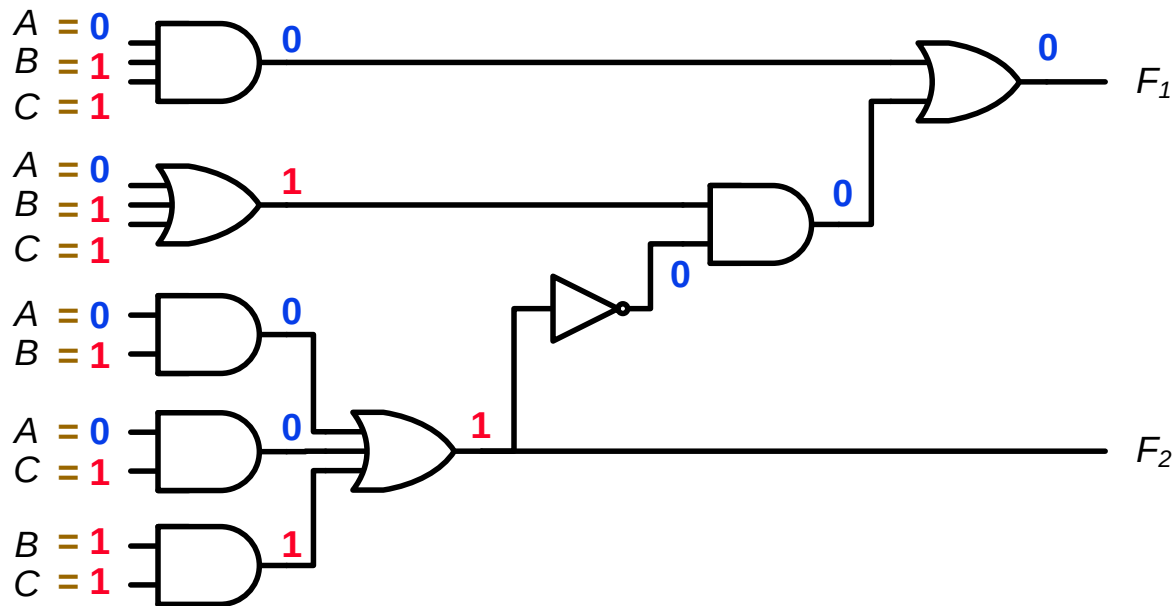
★ Truth Table Approach



A	B	C	F ₁	F ₂
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

Analysis Procedure

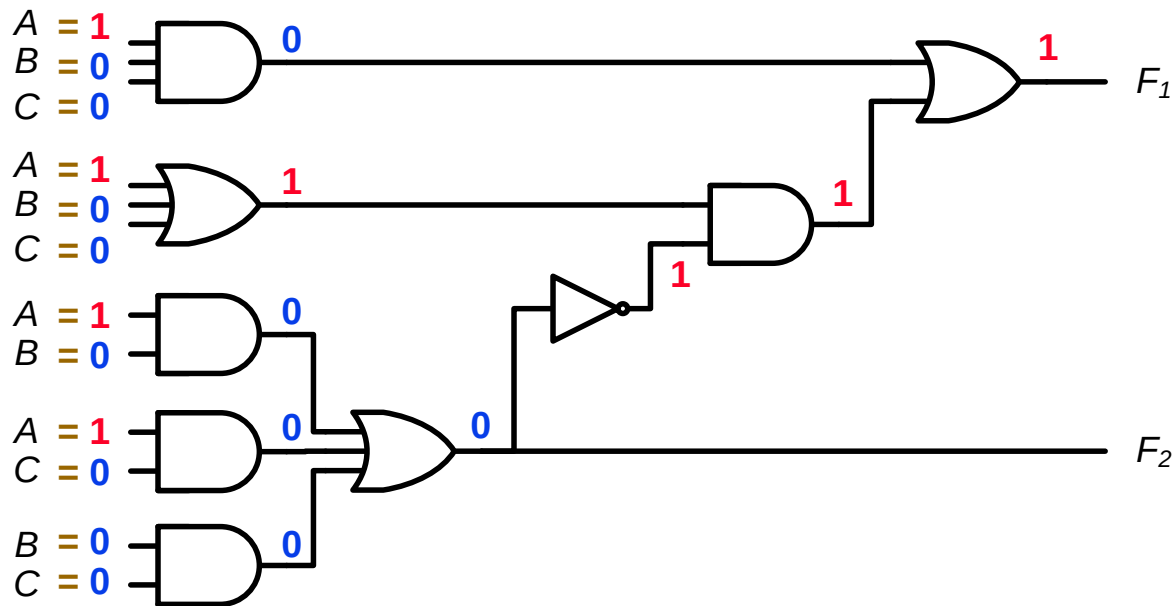
★ Truth Table Approach



A	B	C	F ₁	F ₂
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1

Analysis Procedure

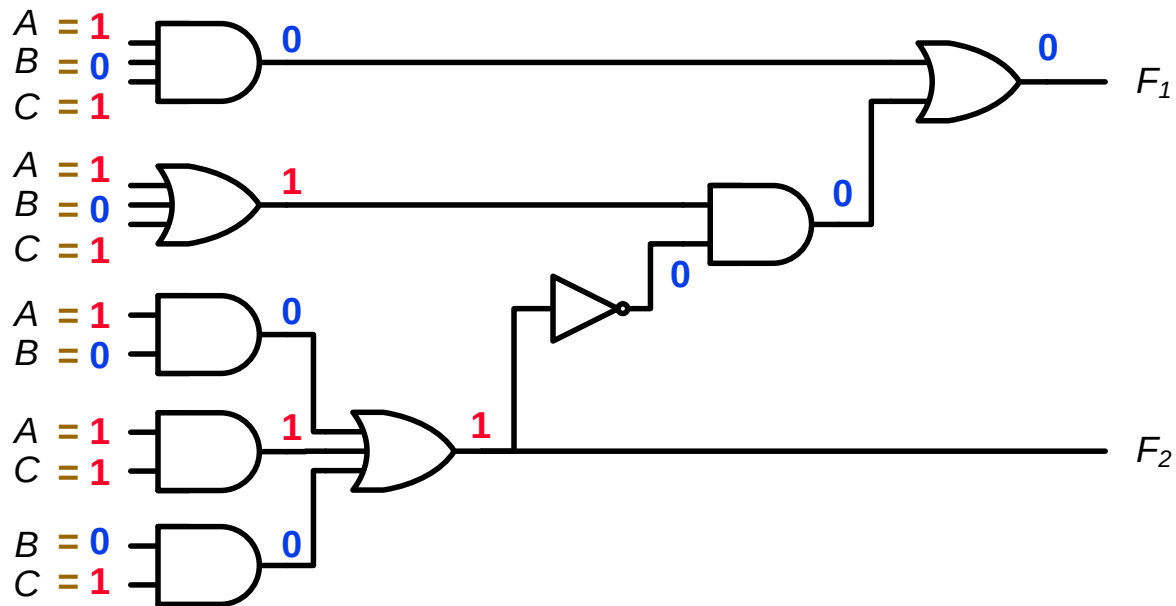
★ Truth Table Approach



A	B	C	F_1	F_2
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0

Analysis Procedure

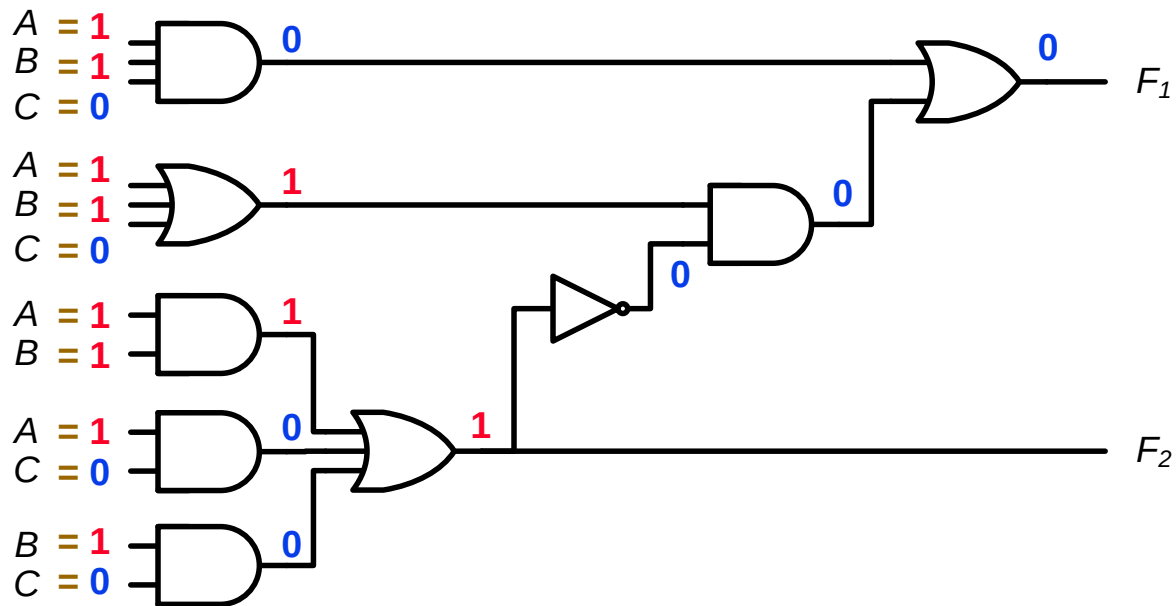
★ Truth Table Approach



A	B	C	F ₁	F ₂
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1

Analysis Procedure

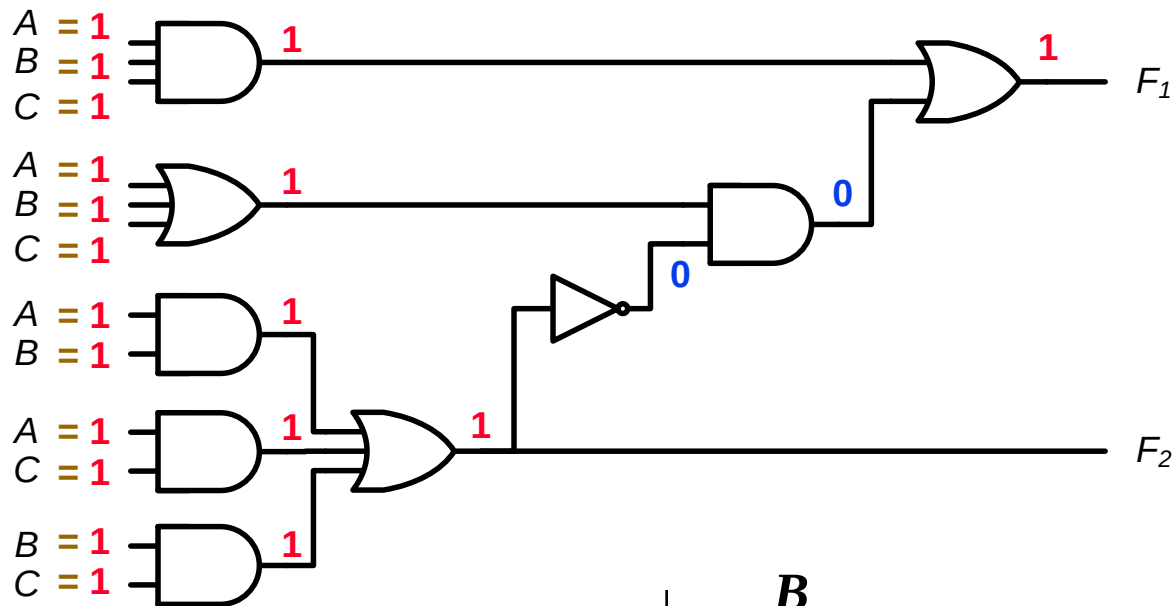
★ Truth Table Approach



A	B	C	F_1	F_2
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1

Analysis Procedure

★ Truth Table Approach



A	B	C	F_1	F_2
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

	B			
	0	1	0	1
A	1	0	1	0
	C			

	B			
	0	0	1	0
A	0	1	1	1
	C			

$$F_1 = AB'C' + A'BC' + A'B'C + ABC$$

$$F_2 = AB + AC + BC$$

Design Procedure

★ Given a problem statement:

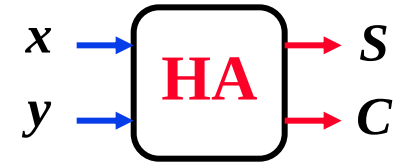
- Determine the number of *inputs* and *outputs*
- Derive the truth table
- Simplify the Boolean expression for each output
- Produce the required circuit

Binary Adder

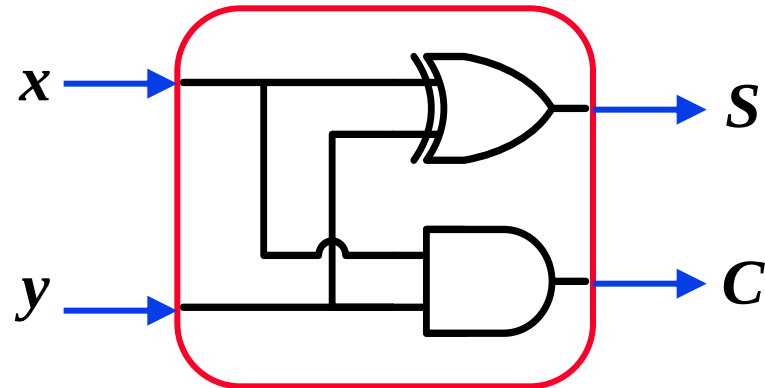
★ Half Adder

- Adds **1-bit** plus **1-bit**
- Produces **Sum** and **Carry**

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



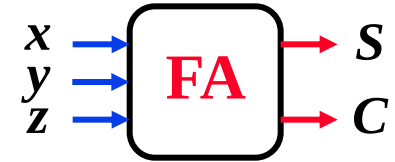
$$\begin{array}{r} x \\ + y \\ \hline C \quad S \end{array}$$



Binary Adder

★ Full Adder

- Adds **1-bit** plus **1-bit** plus **1-bit**
- Produces **Sum** and **Carry**



x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

	y			
	0	1	0	1
x	1	0	1	0
	z			

$$\begin{array}{r}
 x \\
 + y \\
 + z \\
 \hline
 C \quad S
 \end{array}$$

$$S = xy'z' + x'yz' + x'y'z + xyz = x \oplus y \oplus z$$

	y			
	0	0	1	0
x	0	1	1	1
	z			

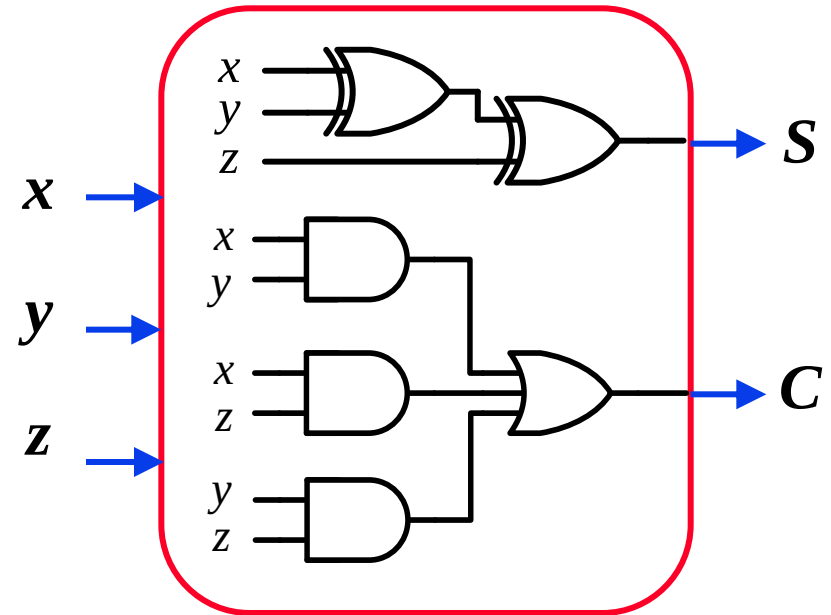
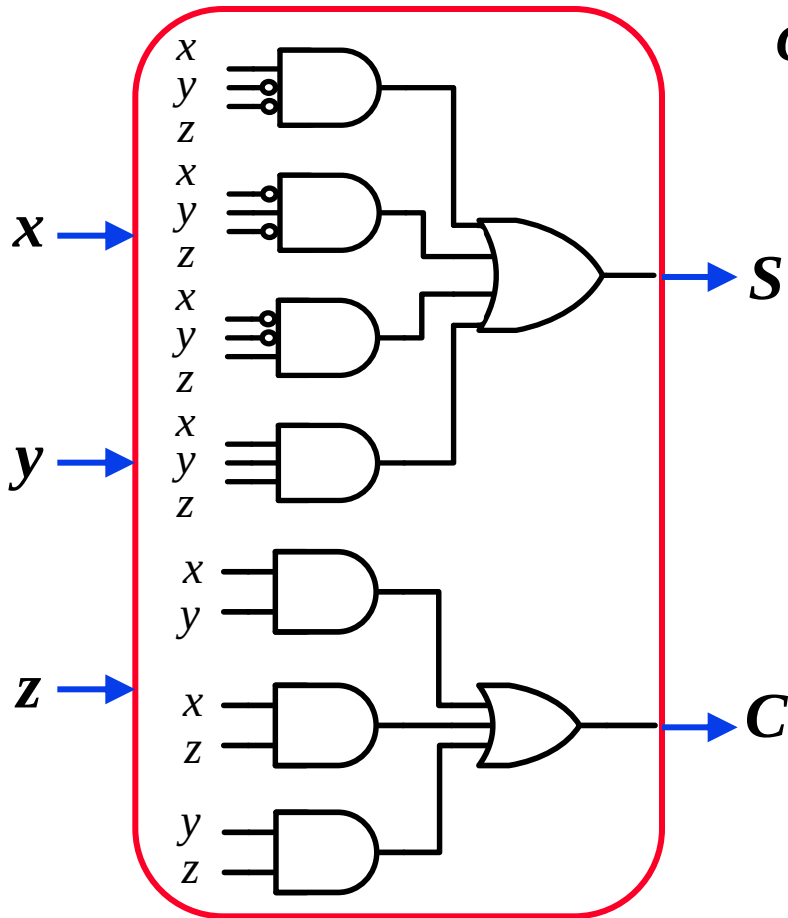
$$C = xy + xz + yz$$

Binary Adder

★ Full Adder

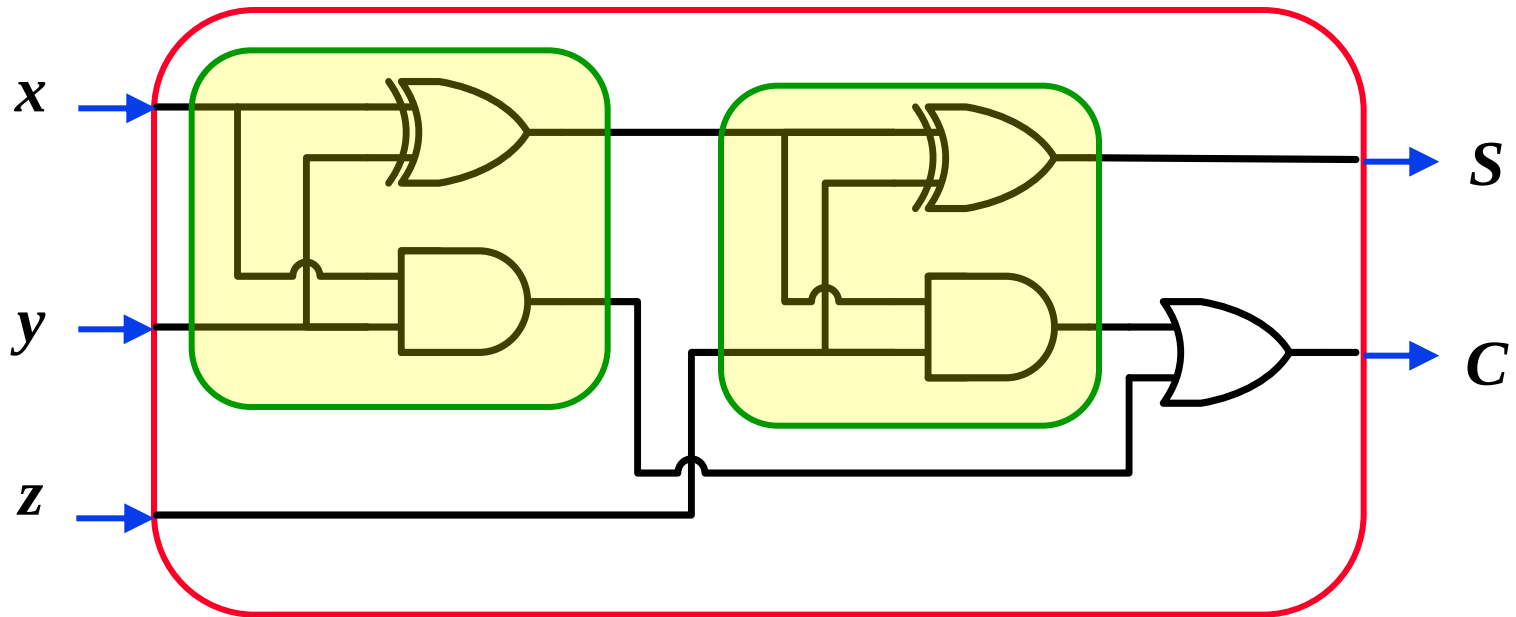
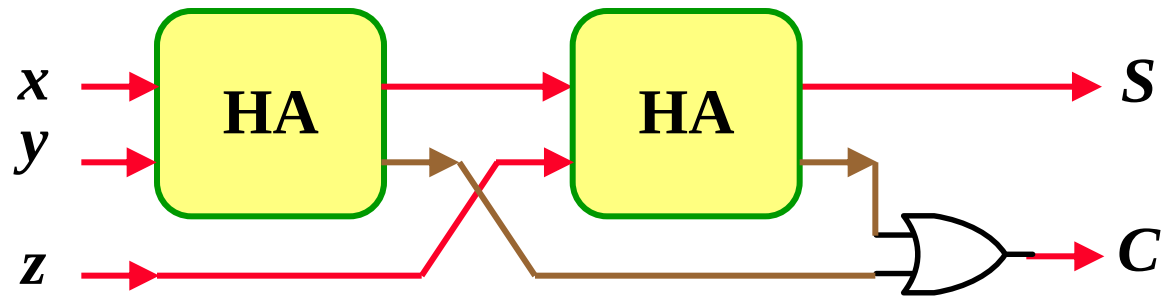
$$S = xy'z' + x'yz' + x'y'z + xyz = x \oplus y \oplus z$$

$$C = xy + xz + yz$$

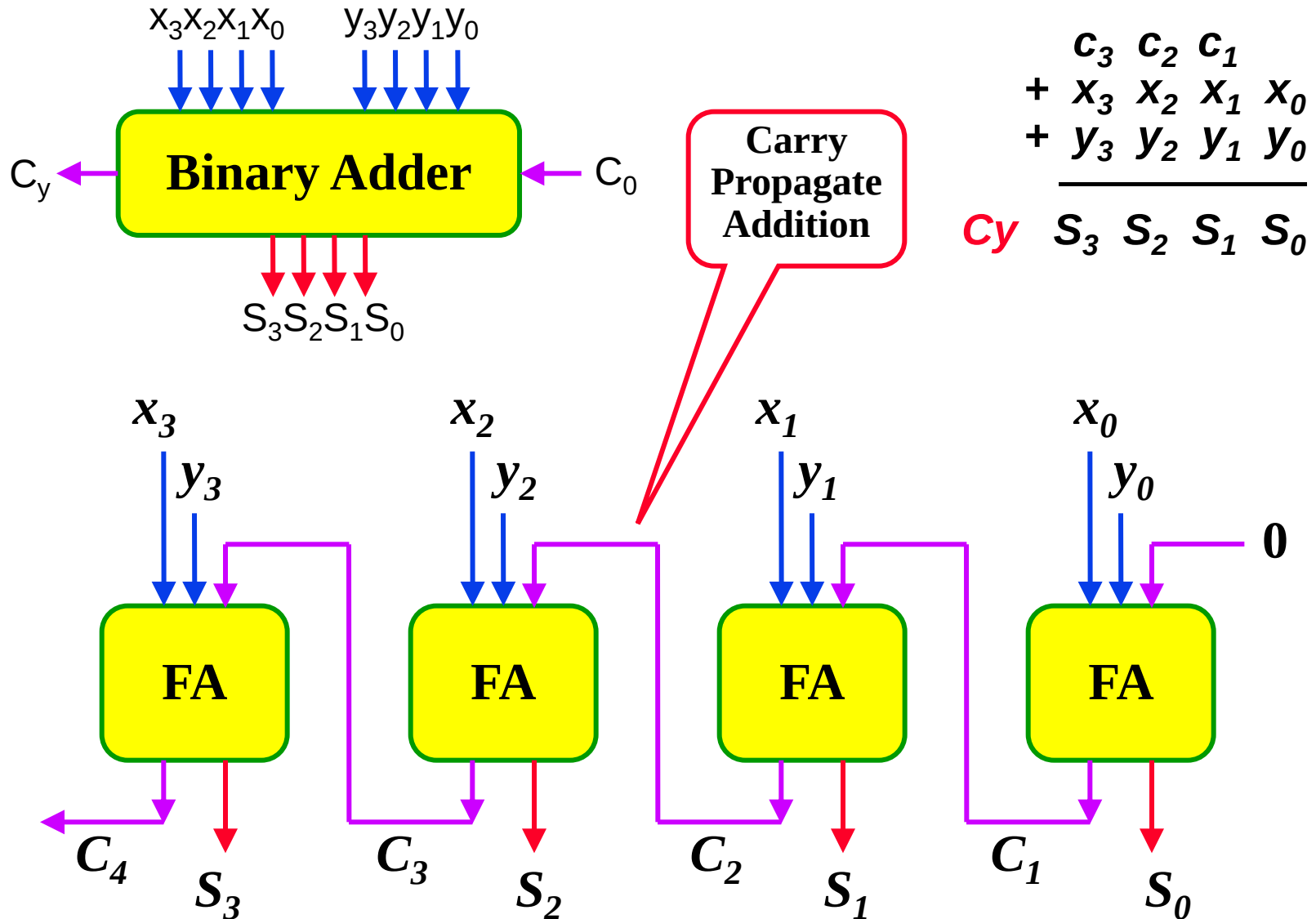


Binary Adder

★ Full Adder



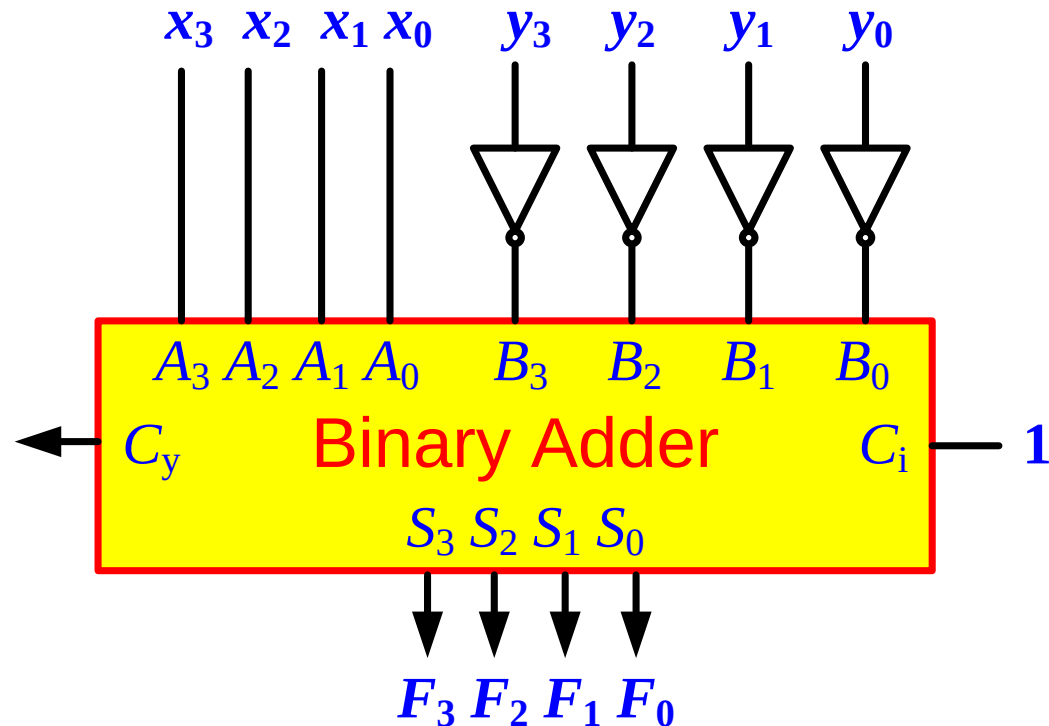
Binary Adder



Binary Subtractor

★ Use 2's complement with binary adder

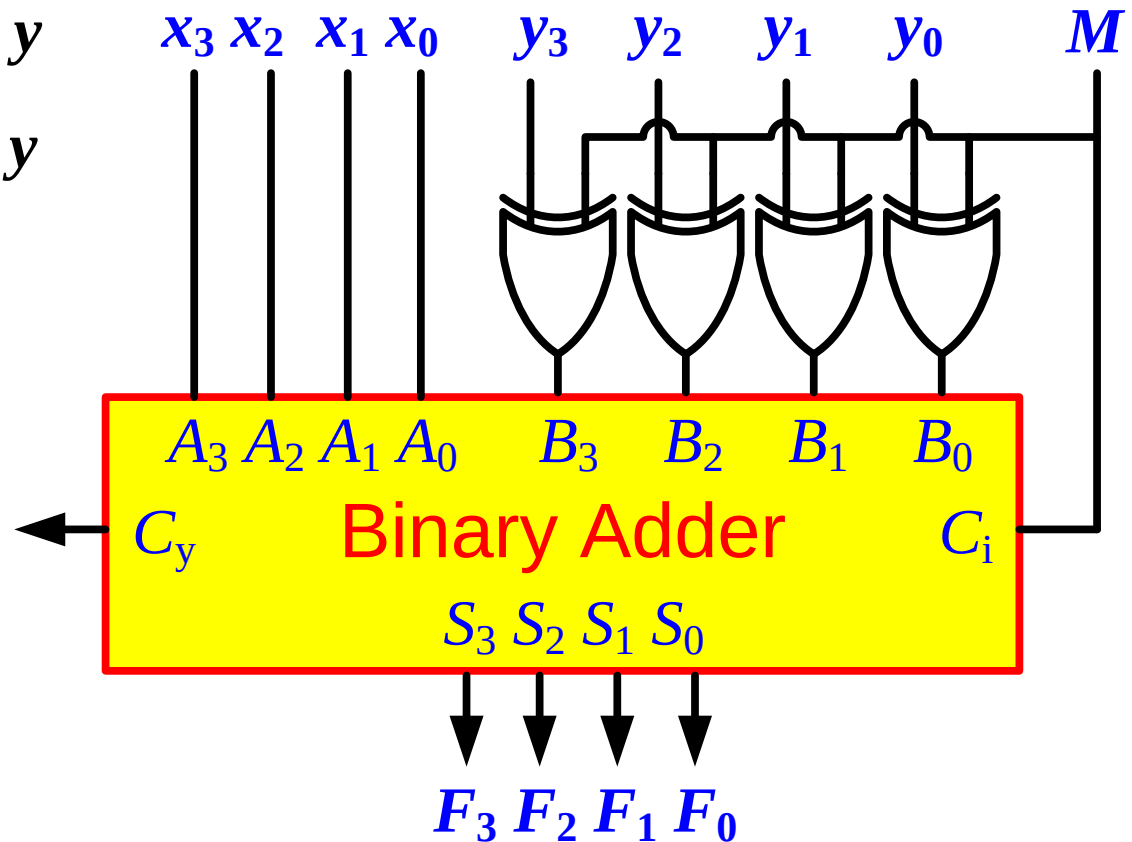
- $x - y = x + (-y) = x + y' + 1$



Binary Adder/Subtractor

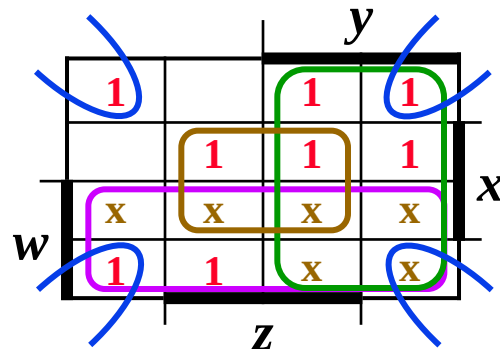
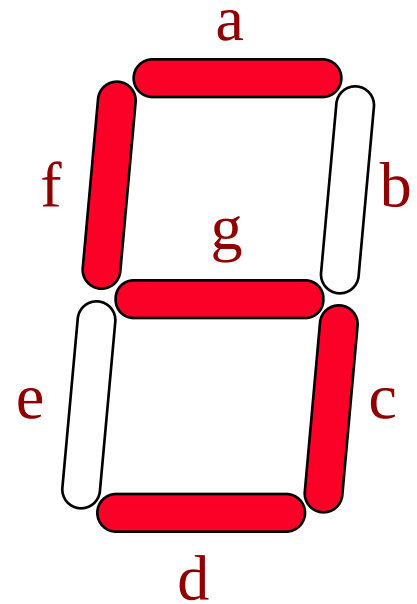
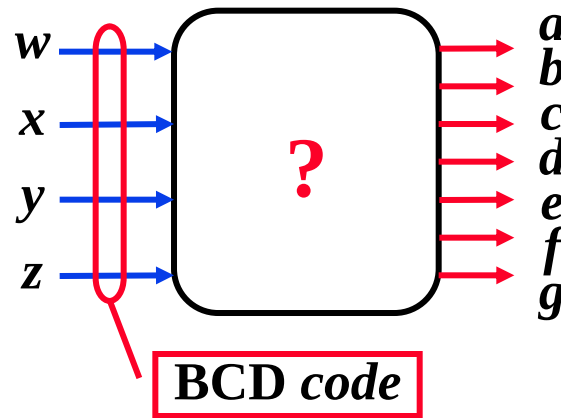
★ M : Control Signal (Mode)

- $M=0 \rightarrow F = x + y$
- $M=1 \rightarrow F = x - y$



Seven-Segment Decoder

<i>w x y z</i>	<i>a b c d e f g</i>
0 0 0 0	1 1 1 1 1 1 0
0 0 0 1	0 1 1 0 0 0 0
0 0 1 0	1 1 0 1 1 0 1
0 0 1 1	1 1 1 1 0 0 1
0 1 0 0	0 1 1 0 0 1 1
0 1 0 1	1 0 1 1 0 1 1
0 1 1 0	1 0 1 1 1 1 1
0 1 1 1	1 1 1 0 0 0 0
1 0 0 0	1 1 1 1 1 1 1
1 0 0 1	1 1 1 1 0 1 1
1 0 1 0	x x x x x x x
1 0 1 1	x x x x x x x
1 1 0 0	x x x x x x x
1 1 0 1	x x x x x x x
1 1 1 0	x x x x x x x
1 1 1 1	x x x x x x x



$$a = w + y + xz + x'z'$$

$$b = \dots$$

$$c = \dots$$

$$d = \dots$$